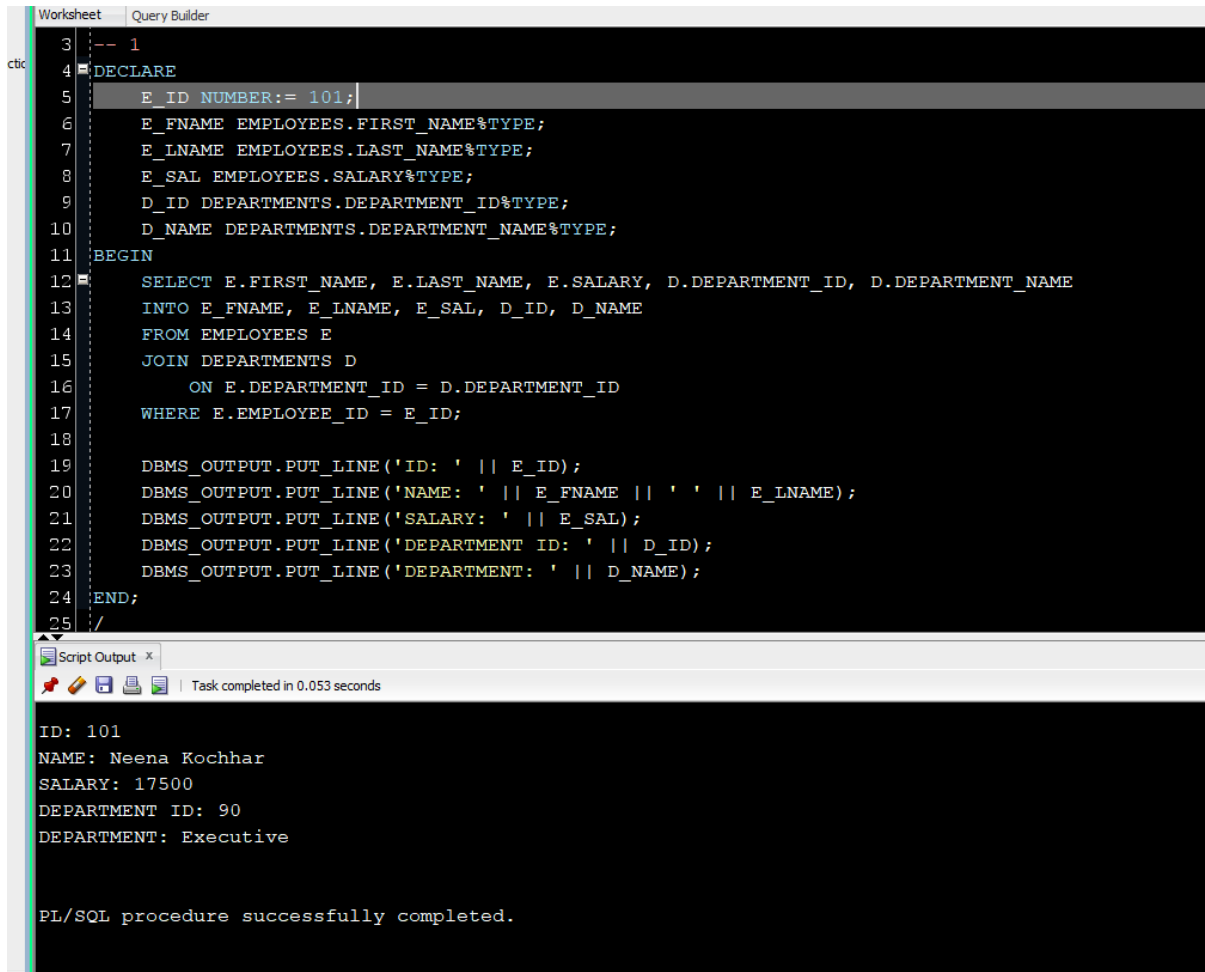


DBS Lab 10 Tasks

1)



The screenshot displays the SQL Developer interface with a 'Worksheet' and 'Query Builder' tab. The main area contains a PL/SQL procedure script. The script declares variables for employee ID, first name, last name, salary, department ID, and department name. It then uses a SELECT statement to retrieve data for employee ID 101, joining the EMPLOYEES and DEPARTMENTS tables. The results are printed using DBMS_OUTPUT.PUT_LINE. The bottom panel shows the 'Script Output' window, which displays the execution results of the procedure.

```
-- 1
3
4 DECLARE
5     E_ID NUMBER:= 101;
6     E_FNAME EMPLOYEES.FIRST_NAME%TYPE;
7     E_LNAME EMPLOYEES.LAST_NAME%TYPE;
8     E_SAL EMPLOYEES.SALARY%TYPE;
9     D_ID DEPARTMENTS.DEPARTMENT_ID%TYPE;
10    D_NAME DEPARTMENTS.DEPARTMENT_NAME%TYPE;
11 BEGIN
12     SELECT E.FIRST_NAME, E.LAST_NAME, E.SALARY, D.DEPARTMENT_ID, D.DEPARTMENT_NAME
13     INTO E_FNAME, E_LNAME, E_SAL, D_ID, D_NAME
14     FROM EMPLOYEES E
15     JOIN DEPARTMENTS D
16     ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
17     WHERE E.EMPLOYEE_ID = E_ID;
18
19     DBMS_OUTPUT.PUT_LINE('ID: ' || E_ID);
20     DBMS_OUTPUT.PUT_LINE('NAME: ' || E_FNAME || ' ' || E_LNAME);
21     DBMS_OUTPUT.PUT_LINE('SALARY: ' || E_SAL);
22     DBMS_OUTPUT.PUT_LINE('DEPARTMENT ID: ' || D_ID);
23     DBMS_OUTPUT.PUT_LINE('DEPARTMENT: ' || D_NAME);
24 END;
25 /
```

Script Output x

Task completed in 0.053 seconds

ID: 101
NAME: Neena Kochhar
SALARY: 17500
DEPARTMENT ID: 90
DEPARTMENT: Executive

PL/SQL procedure successfully completed.

2)

```

27 -- 2
28 BEGIN
29     FOR C IN (
30         SELECT FIRST_NAME, LAST_NAME, JOB_ID, SALARY
31         FROM EMPLOYEES)
32     LOOP
33         DBMS_OUTPUT.PUT(C.FIRST_NAME || ' ' || C.LAST_NAME || ': ');
34         IF C.JOB_ID LIKE '%PROG%' OR C.JOB_ID LIKE '%ENG%' THEN
35             IF C.SALARY > 5000 THEN
36                 DBMS_OUTPUT.PUT_LINE('Eligible for Technical Recognition');
37             ELSE
38                 DBMS_OUTPUT.PUT_LINE('Salary below threshold. ');
39             END IF;
40         ELSE
41             DBMS_OUTPUT.PUT_LINE('Standard Classification');
42         END IF;
43     END LOOP;
44 END;
45 /

```

Script Output x

Task completed in 0.062 seconds

```

Donald OConnell: Standard Classification
Douglas Grant: Standard Classification
Jennifer Whalen: Standard Classification
Michael Hartstein: Standard Classification
Pat Fay: Standard Classification
Susan Mavris: Standard Classification
Hermann Baer: Standard Classification
Shelley Higgins: Standard Classification
William Gietz: Standard Classification

PL/SQL procedure successfully completed.

```

3)

```

47 -- 3
48 BEGIN
49   FOR C IN (
50     SELECT FIRST_NAME, LAST_NAME, SALARY, JOB_ID
51     FROM EMPLOYEES)
52   LOOP
53     DECLARE
54       SAL_LVL VARCHAR2(10) :=
55       CASE
56         WHEN C.SALARY > 10000 THEN 'High'
57         ELSE 'Standard'
58       END;
59     PERFORM VARCHAR2(50) :=
60     CASE
61       WHEN C.JOB_ID LIKE '%MAN%' THEN 'Strategic'
62       ELSE 'Operational'
63     END;
64     BEGIN
65       DBMS_OUTPUT.PUT_LINE(C.FIRST_NAME || ' ' || C.LAST_NAME ||
66       ' | Level: ' || SAL_LVL || ' | Label: ' || PERFORM);
67     END;
68   END LOOP;
69 END;

```

Script Output x

Task completed in 0.048 seconds

```

Susan Mavris | Level: Standard | Label: Operational
Hermann Baer | Level: Standard | Label: Operational
Shelley Higgins | Level: High | Label: Operational
William Gietz | Level: Standard | Label: Operational

```

PL/SQL procedure successfully completed.

4)

```

72 -- 4
73 BEGIN
74   FOR C IN (
75     SELECT LAST_NAME, SALARY, NVL(COMMISSION_PCT, 0) AS COMM
76     FROM EMPLOYEES)
77   LOOP
78     DBMS_OUTPUT.PUT(C.LAST_NAME || ' Status: ');
79     IF (C.SALARY * C.COMM) > 1000 OR C.SALARY > 12000 THEN
80       DBMS_OUTPUT.PUT_LINE('High Performer');
81     ELSE
82       DBMS_OUTPUT.PUT_LINE('Standard Performer');
83     END IF;
84   END LOOP;
85 END;

```

Script Output x

Task completed in 0.053 seconds

```

Grant Status: Standard Performer
Whalen Status: Standard Performer
Hartstein Status: High Performer
Fay Status: Standard Performer
Mavris Status: Standard Performer
Baer Status: Standard Performer
Higgins Status: High Performer
Gietz Status: Standard Performer

PL/SQL procedure successfully completed.

```

5)

```

88 -- 5
89 BEGIN
90   FOR C IN (
91     SELECT NVL(DEPARTMENT_ID, 0) AS DEPARTMENT_ID, SUM(SALARY) AS TOTAL_SAL, AVG(SALARY) AS AVG_SAL
92     FROM EMPLOYEES GROUP BY DEPARTMENT_ID)
93   LOOP
94     DBMS_OUTPUT.PUT('Dept ' || C.DEPARTMENT_ID || ': ');
95     IF C.TOTAL_SAL > 50000 THEN
96       DBMS_OUTPUT.PUT_LINE('Over Budget (Total: ' || C.TOTAL_SAL || ')');
97     ELSE
98       DBMS_OUTPUT.PUT_LINE('Normal Budget: (Avg: ' || ROUND(C.AVG_SAL,2) || ')');
99     END IF;
100   END LOOP;
101 END;
102 /
103

```

Script Output x

Task completed in 0.052 seconds

```

Dept 20: Normal Budget: (Avg: 9500)
Dept 70: Normal Budget: (Avg: 10000)
Dept 110: Normal Budget: (Avg: 10154)
Dept 50: Over Budget (Total: 156400)
Dept 80: Over Budget (Total: 304500)
Dept 40: Normal Budget: (Avg: 6500)
Dept 60: Normal Budget: (Avg: 5760)
Dept 10: Normal Budget: (Avg: 4400)

PL/SQL procedure successfully completed.

```

6)

```

104 -- 6
105 DECLARE
106     TARGET_DEPT EMPLOYEES.DEPARTMENT_ID%TYPE := 100;
107 BEGIN
108     FOR C IN (
109         SELECT * FROM EMPLOYEES
110         WHERE DEPARTMENT_ID = TARGET_DEPT)
111     LOOP
112         DBMS_OUTPUT.PUT_LINE('Report: ' || C.LAST_NAME);
113         DBMS_OUTPUT.PUT_LINE('Job Class: ' || SUBSTR(C.JOB_ID, 1, 2));
114         DBMS_OUTPUT.PUT_LINE('Eligibility: ' ||
115             CASE
116                 WHEN C.SALARY > 7000 THEN 'Qualified'
117                 ELSE 'Pending'
118             END);
119         DBMS_OUTPUT.PUT_LINE('Promotion: ' ||
120             CASE
121                 WHEN C.COMMISSION_PCT IS NOT NULL THEN 'Consider'
122                 ELSE 'N/A'
123             END);
124         DBMS_OUTPUT.PUT_LINE(' ');
125     END LOOP;
126 END;

```

Script Output x

Task completed in 0.041 seconds

```

Report: Popp
Job Class: FI
Eligibility: Pending
Promotion: N/A

```

PL/SQL procedure successfully completed.

7)

```
129 -- 7
130 BEGIN
131   FOR C IN (
132     SELECT LAST_NAME, SALARY, HIRE_DATE, JOB_ID
133     FROM EMPLOYEES)
134   LOOP
135     DBMS_OUTPUT.PUT_LINE(C.LAST_NAME ||
136       ' | Grade: ' ||
137     CASE
138       WHEN C.SALARY > 10000 THEN 'A'
139       ELSE 'B'
140     END ||
141     ' | Exp: ' ||
142     CASE
143       WHEN C.HIRE_DATE < TO_DATE('2015', 'YYYY') THEN 'Senior'
144       ELSE 'Junior'
145     END);
146   END LOOP;
147 END;
148 /
```

Script Output x

Task completed in 0.05 seconds

O'Connell | Grade: B | Exp: Senior
Grant | Grade: B | Exp: Senior
Whalen | Grade: B | Exp: Senior
Hartstein | Grade: A | Exp: Senior
Fay | Grade: B | Exp: Senior
Mavris | Grade: B | Exp: Senior
Baer | Grade: B | Exp: Senior
Higgins | Grade: A | Exp: Senior
Gietz | Grade: B | Exp: Senior

PL/SQL procedure successfully completed.

8)

```
149
150 -- 8
151 BEGIN
152     FOR C IN(
153         SELECT LAST_NAME, DEPARTMENT_ID, SALARY, COMMISSION_PCT
154         FROM EMPLOYEES)
155     LOOP
156         IF (C.DEPARTMENT_ID = 80 AND C.COMMISSION_PCT > 0)
157             OR C.SALARY < 5000 THEN
158             DBMS_OUTPUT.PUT_LINE(C.LAST_NAME || ': Eligible for Bonus');
159         ELSE
160             DBMS_OUTPUT.PUT_LINE(C.LAST_NAME || ': No Bonus');
161         END IF;
162     END LOOP;
163 END;
164 /
```

Script Output x

Task completed in 0.053 seconds

Jones: Eligible for Bonus
Walsh: Eligible for Bonus
Feeney: Eligible for Bonus
OConnell: Eligible for Bonus
Grant: Eligible for Bonus
Whalen: Eligible for Bonus
Hartstein: No Bonus
Fay: No Bonus
Mavris: No Bonus
Baer: No Bonus
Higgins: No Bonus
Gietz: No Bonus

PL/SQL procedure successfully completed.

9)

```
166 -- 9
167 DECLARE
168     AVG_SAL NUMBER;
169 BEGIN
170     SELECT AVG(SALARY) INTO AVG_SAL FROM EMPLOYEES;
171
172     FOR C IN (
173         SELECT LAST_NAME, SALARY, DEPARTMENT_ID
174         FROM EMPLOYEES
175         WHERE SALARY < AVG_SAL)
176     LOOP
177         DBMS_OUTPUT.PUT_LINE('Priority Update: ' || C.LAST_NAME ||
178                               ' | Dept: ' || C.DEPARTMENT_ID);
179     END LOOP;
180 END;
181 /
```

Script Output x

Task completed in 0.054 seconds

Priority Update: Gates | Dept: 50
Priority Update: Perkins | Dept: 50
Priority Update: Bell | Dept: 50
Priority Update: Everett | Dept: 50
Priority Update: McCain | Dept: 50
Priority Update: Jones | Dept: 50
Priority Update: Walsh | Dept: 50
Priority Update: Feeney | Dept: 50
Priority Update: OConnell | Dept: 50
Priority Update: Grant | Dept: 50
Priority Update: Whalen | Dept: 10
Priority Update: Fay | Dept: 20

PL/SQL procedure successfully completed.

10)


```
Worksheet | Query Builder
183 -- 10
184 BEGIN
185     FOR C IN (
186         SELECT LAST_NAME, SALARY, JOB_ID, DEPARTMENT_ID
187         FROM EMPLOYEES)
188     LOOP
189         DECLARE
190             TIER NUMBER :=
191             CASE
192                 WHEN C.SALARY > 12000 THEN 1
193                 WHEN C.SALARY > 6000 THEN 2
194                 ELSE 3
195             END;
196         BEGIN
197             DBMS_OUTPUT.PUT_LINE('Ranking: ' ||
198                 C.LAST_NAME ||
199                 ' | Tier: ' || TIER ||
200                 ' | Job Group: ' || SUBSTR(C.JOB_ID,1, 2));
201         END;
202     END LOOP;
203 END;
204 /
```

Script Output x

Task completed in 0.044 seconds

```
Ranking: Whalen | Tier: 3 | Job Group: AD
Ranking: Hartstein | Tier: 1 | Job Group: MK
Ranking: Fay | Tier: 3 | Job Group: MK
Ranking: Mavris | Tier: 2 | Job Group: HR
Ranking: Baer | Tier: 2 | Job Group: PR
Ranking: Higgins | Tier: 1 | Job Group: AC
Ranking: Gietz | Tier: 2 | Job Group: AC

PL/SQL procedure successfully completed.
```

11)

```
Worksheet Query Builder
206 -- 11
207 BEGIN
208   FOR C IN (
209     SELECT DEPARTMENT_ID, DEPARTMENT_NAME
210     FROM DEPARTMENTS ORDER BY DEPARTMENT_NAME)
211   LOOP
212     DBMS_OUTPUT.PUT_LINE('DEPT: ' || C.DEPARTMENT_NAME);
213     FOR E IN (
214       SELECT LAST_NAME, SALARY, JOB_ID
215       FROM EMPLOYEES
216       WHERE DEPARTMENT_ID = C.DEPARTMENT_ID)
217     LOOP
218       DBMS_OUTPUT.PUT_LINE(
219         ' ' || E.LAST_NAME ||
220         ' | Sal: ' || E.SALARY ||
221         ' | Role: ' || E.JOB_ID);
222     END LOOP;
223     DBMS_OUTPUT.PUT_LINE('-----');
224   END LOOP;
225 END;
226 /
```

Script Output x

Task completed in 0.041 seconds

```
Walsh | Sal: 3100 | Role: SH_CLERK
Feeney | Sal: 3000 | Role: SH_CLERK
OConnell | Sal: 2600 | Role: SH_CLERK
Grant | Sal: 2600 | Role: SH_CLERK
-----
DEPT: Treasury
-----

PL/SQL procedure successfully completed.
```

12)

```
Worksheet | Query Builder
228 -- 12
229 DECLARE
230     E_ID EMPLOYEES.EMPLOYEE_ID%TYPE := 101;
231     E_REC EMPLOYEES%ROWTYPE;
232 BEGIN
233     SELECT * INTO E_REC FROM EMPLOYEES
234     WHERE EMPLOYEE_ID = E_ID;
235     DBMS_OUTPUT.PUT_LINE('Grade: ' ||
236     CASE
237         WHEN E_REC.SALARY > 10000 THEN 'Gold'
238         ELSE 'Silver'
239     END);
240     DBMS_OUTPUT.PUT_LINE('Promotion: ' ||
241     CASE
242         WHEN E_REC.COMMISSION_PCT > 0 THEN 'Eligible'
243         ELSE 'Review Needed'
244     END);
245     EXCEPTION
246         WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('Invalid ID: ' || E_ID);
247 END;
248 /
```

Script Output x

Task completed in 0.045 seconds

PL/SQL procedure successfully completed.

Grade: Gold

Promotion: Review Needed

PL/SQL procedure successfully completed.

13)

```
250 -- 13
251 BEGIN
252     FOR C IN (
253         SELECT LAST_NAME, DEPARTMENT_ID, SALARY, HIRE_DATE
254         FROM EMPLOYEES)
255     LOOP
256         DECLARE
257             NEW_SAL NUMBER := C.SALARY;
258         BEGIN
259             IF C.DEPARTMENT_ID = 90 THEN
260                 NEW_SAL := C.SALARY * 1.10;
261             END IF;
262             DBMS_OUTPUT.PUT_LINE(
263                 C.LAST_NAME ||
264                 ' | Old: ' || C.SALARY ||
265                 ' | New: ' || NEW_SAL);
266         END;
267     END LOOP;
268 END;
269 /
270
```

Script Output x

Task completed in 0.067 seconds

Whalen | Old: 4400 | New: 4400
Hartstein | Old: 13000 | New: 13000
Fay | Old: 6000 | New: 6000
Mavris | Old: 6500 | New: 6500
Baer | Old: 10000 | New: 10000
Higgins | Old: 12008 | New: 12008
Gietz | Old: 8300 | New: 8300

PL/SQL procedure successfully completed.

14)

```
271 -- 14
272 BEGIN
273   FOR C IN (
274     SELECT LAST_NAME, JOB_ID, HIRE_DATE FROM EMPLOYEES)
275   LOOP
276     DECLARE
277       YEARS NUMBER := MONTHS_BETWEEN(SYSDATE, C.HIRE_DATE)/12;
278       CAT VARCHAR2(20);
279       BEGIN
280         CAT :=
281         CASE
282           WHEN YEARS > 10 THEN 'Senior'
283           WHEN YEARS > 5 THEN 'Mid'
284           ELSE 'Junior'
285         END;
286         DBMS_OUTPUT.PUT_LINE(
287           C.LAST_NAME || ' (' || C.JOB_ID ||
288           ') Experience: ' || CAT);
289       END;
290     END LOOP;
291 END;
```

Script Output x

Task completed in 0.047 seconds

Whalen (AD_ASST) Experience: Senior
Hartstein (MK_MAN) Experience: Senior
Fay (MK_REP) Experience: Senior
Mavris (HR_REP) Experience: Senior
Baer (PR_REP) Experience: Senior
Higgins (AC_MGR) Experience: Senior
Gietz (AC_ACCOUNT) Experience: Senior

PL/SQL procedure successfully completed.

15)

```
Worksheet Query Builder
294 -- 15
295 BEGIN
296   FOR C IN (
297     SELECT E.*, D.DEPARTMENT_NAME
298     FROM EMPLOYEES E
299     LEFT JOIN DEPARTMENTS D
300           ON E.DEPARTMENT_ID = D.DEPARTMENT_ID)
301   LOOP
302     DBMS_OUTPUT.PUT_LINE(
303       'EMP: ' || C.LAST_NAME ||
304       ' | DEPT: ' || NVL(C.DEPARTMENT_NAME, 'NONE') ||
305       ' | GRADE: ' ||
306       CASE
307         WHEN C.SALARY > 8000 THEN 'High'
308         ELSE 'Low'
309       END ||
310       ' | BONUS: ' ||
311       CASE
312         WHEN C.COMMISSION_PCT > 0 THEN 'YES'
313         ELSE 'NO'
314       END);
315   END LOOP;
316 END;
317 /
```

Script Output x

Task completed in 0.041 seconds

```
EMP: Fay | DEPT: Marketing | GRADE: Low | BONUS: NO
EMP: Mavris | DEPT: Human Resources | GRADE: Low | BONUS: NO
EMP: Baer | DEPT: Public Relations | GRADE: High | BONUS: NO
EMP: Higgins | DEPT: Accounting | GRADE: High | BONUS: NO
EMP: Gietz | DEPT: Accounting | GRADE: High | BONUS: NO

PL/SQL procedure successfully completed.
```

16)

```

319  -- 16
320  CREATE OR REPLACE PROCEDURE GET_EMP_DETAILS(P_ID IN NUMBER)
321  AS
322      REC EMPLOYEES%ROWTYPE;
323  BEGIN
324      SELECT * INTO REC FROM EMPLOYEES
325      WHERE EMPLOYEE_ID = P_ID;
326      DBMS_OUTPUT.PUT_LINE(
327          'Name: ' || REC.FIRST_NAME ||
328          ' Job: ' || REC.JOB_ID ||
329          ' Salary: ' || REC.SALARY);
330  END;
331  /
332  EXEC GET_EMP_DETAILS(105);
333

```

Script Output x

Task completed in 0.064 seconds

```

Procedure GET_EMP_DETAILS compiled

Name: David Job: IT_PROG Salary: 4800

PL/SQL procedure successfully completed.

```

17)

```

334 -- 17
335 CREATE OR REPLACE PROCEDURE RAISE_SALARY(
336     P_ID IN NUMBER, P_PERCENT IN NUMBER) AS
337     E_NAME EMPLOYEES.LAST_NAME%TYPE;
338     NEW_SAL NUMBER;
339 BEGIN
340     UPDATE EMPLOYEES SET SALARY = SALARY * (1 * P_PERCENT / 100)
341     WHERE EMPLOYEE_ID = P_ID
342     RETURNING LAST_NAME, SALARY INTO E_NAME, NEW_SAL;
343     DBMS_OUTPUT.PUT_LINE(
344     E_NAME || ' Updated Salary: ' || NEW_SAL);
345 END;
346 /
347 EXEC RAISE_SALARY(150, 12);
348
349 -- 18

```

Script Output x

Task completed in 0.056 seconds

Name: David Job: IT_PROG Salary: 4800

PL/SQL procedure successfully completed.

Tucker Updated Salary: 1200

PL/SQL procedure successfully completed.


```

348
349 -- 18
350 CREATE OR REPLACE PROCEDURE LIST_DEPT_EMPS(
351     P_DEPT_ID IN NUMBER) AS
352 BEGIN
353     FOR C IN (
354         SELECT LAST_NAME, JOB_ID, SALARY
355         FROM EMPLOYEES
356         WHERE DEPARTMENT_ID = P_DEPT_ID)
357     LOOP
358         DBMS_OUTPUT.PUT_LINE(
359             C.LAST_NAME ||
360             ' | ' || C.JOB_ID ||
361             ' | ' || C.SALARY);
362     END LOOP;
363 END;
364 /
365 EXEC LIST_DEPT_EMPS(50);
366

```

Script Output x

Task completed in 0.064 seconds

```

Perkins | SH_CLERK | 2500
Bell | SH_CLERK | 4000
Everett | SH_CLERK | 3900
McCain | SH_CLERK | 3200
Jones | SH_CLERK | 2800
Walsh | SH_CLERK | 3100
Feeney | SH_CLERK | 3000
OConnell | SH_CLERK | 2600
Grant | SH_CLERK | 2600

PL/SQL procedure successfully completed.

```

19)

```
367  -- 19
368  CREATE OR REPLACE PROCEDURE DEPT_STATS (
369      P_DEPT_ID IN NUMBER) AS
370      V_COUNT NUMBER;
371      V_SUM NUMBER;
372      V_AVG NUMBER;
373  BEGIN
374      SELECT COUNT(*), SUM(SALARY), AVG(SALARY)
375          INTO V_COUNT, V_SUM, V_AVG
376      FROM EMPLOYEES
377      WHERE DEPARTMENT_ID = P_DEPT_ID;
378
379      DBMS_OUTPUT.PUT_LINE(
380          'Total Employees: ' || V_COUNT ||
381          ' | Total Paid: ' || V_SUM ||
382          ' | Average Pay: ' || ROUND(V_AVG, 2));
383  END;
384  /
385  EXEC DEPT_STATS(50);
386
```

Script Output x

Task completed in 0.067 seconds

PL/SQL procedure successfully completed.

Total Employees: 1 | Total Paid: 6500 | Average Pay: 6500

PL/SQL procedure successfully completed.

Total Employees: 45 | Total Paid: 156400 | Average Pay: 3475.56

PL/SQL procedure successfully completed.

20)

```

387  -- 20
388  CREATE OR REPLACE PROCEDURE LIST_BY_JOB(
389      P_JOB_ID IN VARCHAR2) AS
390  BEGIN
391      FOR C IN (
392          SELECT E.LAST_NAME, D.DEPARTMENT_NAME, E.SALARY
393          FROM EMPLOYEES E
394          JOIN DEPARTMENTS D
395              ON E.DEPARTMENT_ID = D.DEPARTMENT_ID
396          WHERE E.JOB_ID = P_JOB_ID)
397      LOOP
398          DBMS_OUTPUT.PUT_LINE(
399              C.LAST_NAME || ' in ' ||
400              C.DEPARTMENT_NAME || ': $' ||
401              C.SALARY);
402      END LOOP;
403  END;
404  /
405  EXEC LIST_BY_JOB('SA_REP');

```

Script Output x

Task completed in 0.051 seconds

```

FOX in Sales: $9000
Smith in Sales: $7400
Bates in Sales: $7300
Kumar in Sales: $6100
Abel in Sales: $11000
Hutton in Sales: $8800
Taylor in Sales: $8600
Livingston in Sales: $8400
Johnson in Sales: $6200

```

PL/SQL procedure successfully completed.